Terraform Import Guide: Bring Existing Cloud Into IaC

Most cloud engineers inherit infrastructure that was created manually in the cloud portal. Learning how to import resources allows you to safely bring existing cloud assets under Terraform management without downtime or recreation.

What you'll learn:

1. Declarative Import Blocks (Terraform 1.5+)
2. Automatic Code Generation
3. Classic Terminal Import Command
4. State Inspection and Drift Verification

Aman Raj Singh
Cloud Operations Engineer @ iManage
linkedin.com/in/mramanrajsingh

1

TIP 1 OF 4

Declarative Import Blocks (Terraform 1.5+)

Modern Terraform lets you write import statements directly inside your HCL code. This makes the import process repeatable, peer-reviewable, and safely committed to version control. You simply specify the target code address and the real cloud resource ID.

```
import {
  to = aws_s3_bucket.my_legacy_bucket
  id = "my-production-logs-bucket"
}
```

Swipe to tip 2 >> linkedin.com/in/mramanrajsingh

2

TIP 2 OF 4

Automatic Code Generation

Writing HCL code for an existing resource from scratch can be tedious. Using the generate config flag during a plan tells Terraform to read the cloud resource attributes and output a fresh .tf file containing the exact configuration required.

```
$terraform plan -generate-config-out=generated_resources.tf
```

Swipe to tip 3 >> linkedin.com/in/mramanrajsingh

3

TIP 3 OF 4

Classic Terminal Import Command

For older versions of Terraform or quick command-line fixes, you can use the traditional CLI import command. First, declare an empty resource block in your configuration, then run the command passing the target address and real cloud ID.

```
resource "aws_s3_bucket" "app_storage" {}

# Run in your terminal:
terraform import aws_s3_bucket.app_storage app-storage-bucket-id-123
```

Swipe to tip 4 >> linkedin.com/in/mramanrajsingh

4

TIP 4 OF 4

State Inspection and Drift Verification

After importing, inspect the newly captured state to confirm all attributes are recorded. Use state show to read property values, update your local configuration file to match, and run terraform plan to confirm zero pending changes.

```
$terraform state show aws_s3_bucket.app_storage
 terraform plan
```

See Key Takeaways on next page >> linkedin.com/in/mramanrajsingh

Key Takeaways

+ Always back up your `terraform.tfstate` file prior to performing CLI imports.
+ Use modern `import {}` blocks so imports are tracked in version control.
+ Run `terraform plan` after importing to verify code matches real infrastructure.
+ Clean up generated code files by removing read-only or default parameters.
+ Import cloud resources incrementally rather than trying to import everything at once.

Watch Out For

! Running classic `terraform import` without defining a resource block in your code first.
! Assuming `terraform import` writes local .tf code automatically (it only updates state by default).
! Executing `terraform apply` before running `terraform plan` to confirm there are no unintended destructions.

Follow for daily Cloud & DevOps guides!

linkedin.com/in/mramanrajsingh